CHAPTER

15 Failure detectors

15.1 Introduction

This chapter deals with the design of fault-tolerant distributed systems. It is widely known that the design and verification of fault-tolerent distributed systems is a difficult problem. Consensus and atomic broadcast are two important paradigms in the design of fault-tolerent distributed systems and they find wide applications. Consensus allows a set of processes to reach a common decision or value that depends upon the initial values at the processes, regardless of failures. In atomic broadcast, processes reliably broadcast messages such that they agree on the set of messages delivered and the order of message deliveries.

This chapter focuses on solutions to consensus and atomic broadcast problems in asynchronous distributed systems. In asynchronous distributed systems, there is no bound on the time it takes for a process to execute a computation step or for a message to go from its sender to its receiver. In an asynchronous distributed system, there is no upper bound on the relative processor speeds, execution times, clock drifts, and delay during the transmission of messages although they are finite. This is mainly casued by unpredictable loads on the system that causes asynchrony in the system and one cannot make any timing assumptions of any types. On the other hand, synchronous systems are characterized by strict bounds on the execution times and message transmission delays.

The asynchronous model of distributed system has simpler semantics when compared to synchronous model. Applications based on the asynchronous model are easily portable because there are no strict timing assumptions to take care of. The asynchronous model of distributed systems is very popular and has attracted lot of attention due to these reasons. Inspite of the attractiveness of asynchronous distributed systems, it is well known that consensus, atomic broadcast, and several other reliable broadcast problems cannot be solved deterministically even for a single process failure due to the unbounded timing characteristics. The main cause of this impossibility result is that it is very

difficult to determine in asynchronous systems whether a process has failed or is simply taking a long time for execution; so it is difficult to deal with failures in these systems. On the other hand, in synchronous systems due to strict timing constraints, failures can easily be detected.

The asynchronous model of distributed systems is widely used, and such systems are prone to failures. Thus, detection and/or prevention of failures in these systems is of vital importance. The detection of process failures is a crucial task in the design of fault tolerant distributed systems. Detection of crashed processes is especially difficult in asynchronous systems as it is impossible to determine whether a process has really crashed or is very slow (as there are no timing constraints present).

In this chapter, we discuss the concept of unreliable failure detectors to deal with the impossibility results in asynchronous distributed systems with crash failures. Basically, the asynchronous model of computation is extended with a failure detection mechanism that is prone to errors in the sense that a process can brand another process as crashed even though the process is running. We study failure detectors in asynchronous distributed systems. We investigate two major problems faced in asynchronous distributed environments, namely, consensus and atomic broadcast. We study several solutions for these problems.

15.2 Unreliable failure detectors

Chandra and Toueg [3] introduced the concept of unreliable failure detectors and showed how unreliable failure detectors can be used to solve two fundamental paradigms of asynchronous distributed systems with crash failures, namely, consensus and atomic broadcast.

15.2.1 The system model

We consider asynchronous distributed systems in which there is no bound on message delay, clock drift, or the time taken to execute a step. The system consists of a finite set of n processes, $Q = \{p_1, p_2, ..., p_n\}$. Each pair of processes is connected by a reliable communication channel. A process can fail by crashing only, i.e., by prematurely halting. A process behaves correctly (i.e., according to its specification) until it crashes.

A discrete global clock is assumed, and the range of the clock's ticks, Φ, is the set of natural numbers. The global clock is used for the sake of simplicity of presentation and reasoning and is not accessible to the processes.

A process p_i is said to crash at time t if p_i does not perform any action after time t. Process failures are permanent; once a process crashes, it does not recover. A *correct* process is a process that has not crashed.

Informally, a run is an infinite execution of the system. Given any run σ, *Crashed*(t, σ) is the set of processes that have crashed by time t and $Up(t, \sigma)$ is the set of processes that are correct (i.e., have not crashed) by time t, that is, $Up(t, \sigma) = Q - Crashed(t, \sigma)$. $Crashed(\sigma)$ is the set of processes that have crashed in a run σ and is equal to $\bigcup_t Crashed(t, \sigma)$. $Up(\sigma)$ is the set of processes that are correct in a run σ and is equal to $Q - Crashed(\sigma)$. If a process $p \in Crashed(\sigma)$, we say that p is a faulty process in σ. If a process $p \in Up(\sigma)$, we say that p is a correct process in σ. We consider only execution runs where at least one process is correct.

Failure patterns and environments

A failure pattern is a function F from Φ to 2^Q, where $F(t)$ denotes the set of processes that have crashed through time t. An environment E is a set of failure patterns. Environments describe the crashes that can occur in a system. In general, we consider the environments that contain all possible failure patterns, i.e., there is no bound on the number of processes that crash.

Each process p_i has a local failure detector module of D, denoted by D_i. Associated with each failure detector D is a range R_D of values output by the failure detector. A failure detector history H with range R is a fuction H from $\Omega X \Phi$ to R. $D(F)$ denotes the set of possible failure detector histories permitted for the failure pattern F, i.e., each history represents a possible behavior of D for the failure pattern F. For any failure detector D, any failure pattern F, and any history H in $D(F)$, $H(p_i, t)$ is the set of processes suspected by process p_i at time t.

15.2.2 Failure detectors

A failure detector D is a distributed oracle that gives hints about failure patterns. Each process p_i in the distributed environment has its own local failure detector D_i, which monitors all other processes and maintains a list of processes, currently p_i suspects to have crashed. The suspicion is based on relative timeouts of other processes at p_i.

Thus, a failure detector D as the vector $D = \langle D_{p1}, D_{p2}, ...D_{pn} \rangle$, where D_i is the failure detector module at process p_i, that outputs the set of processes that it currently suspects to have crashed. Formally, a failure detector is a function "from time and the set of all runs" to 2^Q. $D_p(t, \sigma)$ is the set of processes that are suspected to have crashed by p's failure detector module at time t in run σ. If $q \in D_p(t, \sigma)$, we say that p suspects q at time t in run σ. After a process crashes, it is immaterial what its failure detector module indicates. We formalize this by assuming that if $p \in Crashed(t, \sigma)$, then $D_p(t, \sigma) = \phi$.

The failure detectors can make mistakes, i.e., a correct process may be added to the list of suspects and can later be removed if the failure detector realizes that it was a mistake. Thus, a failure detector may continually add

and remove processes from its list of suspects. Processes can be added and removed from the list of suspects by each failure detector module any number of times. At any time, failure detector modules at two processes may have different lists of suspects.

It should be noted that the addition of a correct process to the list of suspects by any other process or by all other processes should not prevent this process from behaving correctly, according to its specifications.

15.2.3 Completeness and accuracy properties

Chandra and Toueg [3] classified failure detectors in terms of their completeness and accuracy properties. Informally, completeness requires that a failure detector eventually suspects all processes that have crashed and accuracy resticts the mistakes a failure detector can make (i.e., a correct process suspect another correct process). They define two types of completeness and four types of accuracy properties, giving rise to eight classes of failure detectors.

Chandra and Toueg [3] introduced the concept of reducibility among failure detectors. Informally, a failure detector D is *reducible* into another failure detector D' if there exists a distributed algorithm that can transform D into D'. In this case, any problem that can be solved using D' can also be solved using D. If two failure detectors are reducible to each other, they are said to be *equivalent*.

Chandra and Toueg [3] put failure detectors into eight classes and ordered them into a hierarchy according to the reducibility relationship. In this hierarchy, some failure detectors can solve the consensus problem with any number of process failures, while others require a certain number of correct processes to solve the consensus problem. This requirement and the boundary where this requirement becomes necessary have been clearly specified.

We now define completeness and accuracy properties of a failure detector.

Completeness

Definition 15.1 (Completeness) There is a time after which every process that has crashed is permanently suspected by a correct process.

Completeness can be of two types:

- **Strong completeness** Eventually every process that crashes is permanently suspected by every correct process. Notationally,

$$\forall \sigma, \forall p \in Crashed(\sigma), \forall q \in Up(\sigma), \exists t \ such \ that \ \forall t' \geq t : p \in D_q(t', \sigma).$$

- **Weak completeness** Eventually every process that crashes is permanently suspected by some correct process. Notationally,

$$\forall \sigma, \forall p \in Crashed(\sigma), \exists q \in Up(\sigma), \exists t \ such \ that \ \forall t' \geq t : p \in D_q(t', \sigma).$$

Note that completeness by itself may not be of much use. For example, a failure detector may satisfy the strong completeness property by having every process permanently suspect all other processes. Such a failure detector is useless because it provides no information about actual failures. Thus, a failure detector must satisfy some accuracy property to be useful. We define this property next.

Accuracy

Definition 15.2 (Accuracy) *There is a time after which a correct process is never suspected by any correct process.*

There are two types of accuracy property:

- **Strong accuracy** Correct processes are never suspected by any correct process. Formally,

$$\forall \sigma, \forall t, \forall p, q \in Up(t, \sigma) : p \notin D_q(t, \sigma).$$

Since in any practical system it is extremely difficult to achieve accuracy, we weaken it as follows:

- **Weak accuracy** Some correct process is never suspected by any correct process. Formally,

$$\forall \sigma, \exists p \in Up(\sigma), \forall t, \forall q \in Up(t, \sigma) : p \notin D_q(t, \sigma).$$

We collectively refer to strong accuracy and weak accuracy as the *perpetual accuracy* properties because these properties hold all the time. Note that even weak accuracy is difficult to achieve, because a failure detector (even at a correct process) may suspect a correct process and then later correct its mistake. The weak accuracy property does not permit this. Thus, we further weaken the accuracy requirement and allow failure detectors that may suspect a correct process at some points in the run, but they *eventually* satisfy the strong and weak accuracy properties.

Eventual accuracy

Definition 15.3 (Eventual accuracy) *We need not require accuracy property to be satisfied by each process at all the time. Instead, we require the accuracy property to be eventually satisfied.*

There are two types of eventual accuracy:

- **Eventual strong accuracy** There is a time after which correct processes are not suspected by any correct process. Formally,

$$\forall \sigma, \exists t, \forall t' \geq t, \forall p, q \in Up(t', \sigma) : p \notin D_q(t', \sigma).$$

- **Eventual weak accuracy** There is a time after which some correct process is not suspected by any correct process. Formally,

$$\forall \sigma, \exists t, \forall t' \geq t, \exists p \in Up(\sigma), \forall q \in Up(\sigma) : p \notin D_q(t', \sigma).$$

We collectively refer to eventual strong accuracy and eventual weak accuracy as the *eventual accuracy* properties because these properties hold eventually.

15.2.4 Types of failure detectors

Based on types of accuracies and completeness defined above, failure detectors can be classified into the following categories:

- **Perfect failure detectors** (P) Failure detectors that satisfy the strong completeness and the strong accuracy properties are called perfect failure detectors.
- **Eventually perfect failure detectors** ($\diamond P$) Failure detectors that satisfy the strong completeness and the eventual strong accuracy properties are called eventually perfect failure detectors.
- **Strong failure detectors** (S) Failure detectors that satisfy the strong completeness and the weak accuracy properties are called strong failure detectors.
- **Eventually strong failure detectors** ($\diamond S$) Failure detectors that satisfy the strong completeness and the eventual weak accuracy properties are called eventually strong failure detectors.
- **Weak failure detectors** (W) Failure detectors that satisfy the weak completeness and the weak accuracy properties are called weak failure detectors.
- **Eventually weak failure detectors** ($\diamond W$) Failure detectors that satisfy the weak completeness and the eventual weak accuracy properties are called eventually weak failure detectors.
- Another class of failure detector is the one that satisfies weak completeness and strong accuracy properties. This class is denoted by ϑ.
- The last class is the set of failure detectors that satisfy weak completeness and eventually strong accuracy properties. This class is denoted by $\diamond\vartheta$.

15.2.5 Reducibility of failure detectors

A failure detector D is reducible to another failure detector D' if there is an algorithm that transforms a failure detector D into another failure detector D'. A natural question is: what does it mean that an algorithm transforms D into D'? An algorithm $T_{D \rightarrow D'}$ transforms a failure detector D into another failure detector D' if and only if for every run R of $T_{D \rightarrow D'}$ under a failure pattern F using D, $output^R \in D'(F)$, where $output^R$ is the output of run R using failure detector D and $D'(F)$ denotes the set of histories of failure detector D' for

failure pattern F. That is, variable $output_p$ at process p emulates the output of D'. Thus, $T_{D \to D'}$ can emulate D' using D. $T_{D \to D'}$ need not emulate all failure detector histories of D'; however, all failure detector histories it emulates must be histories of D'. Algorithm $T_{D \to D'}$ is called the *reduction algorithm*.

Given a reduction algorithm $T_{D \to E}$, any problem that can be solved using E, can also be solved using D. We illustrate this with an example: suppose a given algorithm A requires failure detector E, but only failure detector D is available. We can execute A using failure detector D as follows. Concurrently with A, processes run $T_{D \to E}$ to transform D to E. Algorithm A is modified at process p as follows: whenever A requires that p queries its failure detector module, p reads the current value of $output_p$, which is concurrently maintained by $T_{D \to E}$.

Since $T_{D \to E}$ is able to use D to emulate E, D must provide at least as much information about process failures as E does. Thus, if there is an algorithm $T_{D \to E}$ that transforms D into E, we say that E is weaker than D and denote it by $D \sqsubseteq E$. Note that $\sqsubseteq$ is a transitive relation. If $D \sqsubseteq E$ and $E \sqsubseteq D$, then we say that D and E are *equivalent* and denote it by $D \equiv E$.

If D and ε are two classes of failure detectors and there exists an algorithm $T_{D \to E}$ that can transform every failure detector $D \in D$ into a failure detector $E \in \varepsilon$, then we say that the class of failure detectors D is reducible to the class of failure detectors ε and this is denoted by $D \sqsubseteq \varepsilon$. In this case, ε is weaker than D. If $D \sqsubseteq \varepsilon$ and $\varepsilon \sqsubseteq D$, then D and ε are equivalent and this is denoted by $D \equiv \varepsilon$.

From a trivial reduction algorithm, where each process p periodically writes the current output of its failure detector module into $output_p$, the following relations between the classes of failure detectors are obvious:

Observation 15.1 $P \sqsubseteq \vartheta$, $S \sqsubseteq W$, $\diamond P \sqsubseteq \diamond\vartheta$, $\diamond S \sqsubseteq \diamond W$.

15.2.6 Reducing weak failure detector *W* to a strong failure detector *S*

In Algorithm 15.1, we give a reduction algorithm $T_{D \to D'}$ (due to Chandra and Toueg [3]) that transforms any given failure detector D that satisfies weak completeness, into a failure detector D' that satisfies strong completeness. D' satisfies the same accuracy property that D satisfies. Thus, this algorithm strenghtens the completeness while preserving the accuracy.

Informally, the conversion of any weak failure detector W to a strong failure detector S is as follows: initially, for every process p, $output_p$ is set to null. (Recall that $output_p$ is the variable emulating the output of the failure detector module D'_p.) Every process p periodically sends $(p, suspects_p)$ to every process, where $suspects_p$ denotes the set of processes that p suspects according to its failure detector module D_p. When a process p recieves a message $(q, suspects_q)$ from a process q, process p adds the suspect list of process q, $suspects_q$, to its output, $output_p$, and removes the process q from its output as it is a correct process.

Every process p executes the following:
$output_p \leftarrow \phi$
cobegin
||Task 1: repeat forever
$suspects_p \leftarrow D_p$ $\{p$ queries its local failure detector module $D_p\}$
$send(p, suspects_p)$ to all other processes.
||Task 2: when receive $(q, suspects_q)$ for a process q
$output_p \leftarrow (output_p \cup suspects_q) - \{q\}$ $\{output_p$ emulates $E_p\}$
coend

Algorithm 15.1 Transforming weak completeness to strong completeness [3].

A correctness argument

The correctness proof of the algorithm involves showing the following three properties:

1. It transforms weak completeness into strong completeness.
2. It preserves the perpetual accuracy.
3. It preserves the eventual accuracy.

We show these properties in the following three lemmas.

Lemma 15.1 *Let p be any process that crashes. If eventually some correct process permanently suspects p in H_D, then eventually all correct processes permanently suspect p in $output^R$, where H_D is the history of failure detector D and $output^R$ is the output of an arbitrary run R using failure detector D.*

Since process p crashes, there is a time t' after which no process recieves a message from p. Suppose there is a correct process q that permanently suspects p in H_D after time t. Consider the execution of *task* 1 by process q after time $t_p = max(t, t')$. Process q sends a message $(q, suspects_q)$ such that $p \in suspects_q$ to all processes. Eventually, every correct process recieves $(q, suspects_q)$ and adds p to output (in *task* 2). Since no correct process recieves any messages from p after time t' and $t_p \geq t'$, no correct process removes p from its *output* after t_p. Thus, there is a time after which every correct process permanently suspects p in $output^R$.

Lemma 15.2 *Let p be any process. If no process suspects p in H_D before time t, then no process suspects p in $output^R$ before time t.*

Suppose there is a time t before which no process suspects process p in H_D. Thus, no process sends a message of type $(-, suspects)$ such that $p \in suspects$ before time t. Thus, no process q adds p to $output_q$ before time t.

Lemma 15.3 *Let p be a correct process. If there is a time after which no correct process suspects p in H_D, then there is a time after which no correct process suspects p in $output_R$.*

Suppose there is a time t after which no correct process suspects p in H_D. Thus, all processes that suspect p after time t eventually crash. Thus, there is time after which no process will send messages of type (–, *suspects*) such that $p \in suspects$. Thus, there is a time t' after which no correct process recieves a message of type (–, *suspects*) such that $p \in suspects$. Let q be a correct process. We need to show that there is a time after which q does not suspect p *in* $output^R$. Consider the execution of *task* 1 by process p after time t'. Process p sends the message $(p, suspects_p)$ to q. When q receives this message, it removes p from $output_q$ if p is present in $output_q$ (*task* 2). Note that q does not receive any messages of type $(–, suspects)$ such that $p \in suspects$ after time t'; therefore, q does not add p to $output_q$ after time t'. Thus, there is a time after which q does not suspect p in $output^R$.

Theorem 15.1 $\vartheta \sqsubseteq P, W \sqsubseteq S, \diamond\vartheta \sqsubseteq \diamond P$ *and* $\diamond W \sqsubseteq \diamond S$.

Proof Let D be any failure detector in ϑ, W, $\diamond\vartheta$, or $\diamond$ W. We show that $T_{D \rightarrow E}$ transforms D into a failure detector E in P, S, $\diamond P$, or $\diamond S$. Since D satisfies weak completeness, E satisfies strong completeness (from Lemma 15.1). We now argue that D and E have the same accuracy properties. If D is in ϑ or W, then D and E have the same accuracy property (from Lemma 15.2). If D is in $\diamond\vartheta$ or $\diamond W$, then D and E have the same accuracy property (from Lemma 15.3).

Thus, we have:

$$\vartheta \sqsubseteq P, W \sqsubseteq S, \diamond\vartheta \sqsubseteq \diamond P \text{ and } \diamond W \sqsubseteq \diamond S. \qquad \square$$

From Theorem 15.1 and Observation 15.1, we have the following result:

$$P \equiv \vartheta, S \equiv W, \diamond P \equiv \diamond\vartheta, \text{ and } \diamond S \equiv \diamond W.$$

A significance of this result is that if we solve a problem for the four failure detectors with strong completeness, the problem is automatically solved for the remaining four failure detectors.

15.2.7 Reducing an eventually weak failure detector $\diamond W$ to an eventually strong failure detector $\diamond S$

Algorithm 15.2 gives an algorithm that converts any eventually weak failure detector $D \in \diamond W$ into an eventually strong failure detector $E \in \diamond S$. Q is the set of all processes.

At process p, variable $suspected_p(r, q)$ denotes how many times process q has suspected process r and variable $refuted_p(r, q)$ denotes how many

times process r has refuted process q. Both variables are initialized to zero. S_p denotes the suspect list of process p.

Process p runs the following:

for all $q, r \in Q$
{Number of times q suspected r according to p}
$suspected_p(r, q) \leftarrow 0$
{Number of times r refuted q according to p}
$refuted_p(r, q) \leftarrow 0$

cobegin
||Task 1: repeat forever
if ($r \in D_p$ and $refuted_p(r, p) \leq suspected_p(r, p)$) then
  p rbcasts (p, *suspects*, r, $refuted_p(r, p) + 1$)

||Task 2: when p rbdelivers (q, *suspects*, r, k)
  $suspected_p(r, q) \leftarrow$ k
  if $p = r$ then p rbcasts (p, *refutes*, q, k)

||Task 3: when p redelivers (r, *refutes*, q, k)
  $refuted_p(r, q) \leftarrow k$

||Task 4: repeat forever
  for all processes r
    if $\exists\, q : suspected_p(r, q) > refuted_p(r, q)$
      then $S_p \leftarrow S_p \bigcup \{r\}$
    else $S_p \leftarrow S_p - \{r\}$
coend

Algorithm 15.2 An algorithm to reduce an eventually weak failure detector into an eventually strong failure detector [3].

An explanation of the algorithm

The algorithm consists of four tasks.

In task 1, a process p continuously performs the following for every process r that it suspects according to its failure detector module D_p: if the number of times process r is suspected by p is greater than the number of times r has refuted p, then p broadcasts a suspect message that contains the incremented refuted value.

In task 2, when process p receives a suspect message (q, *suspects*, r, k) from a process q, it updates $suspected_p(r, q)$ to k. If process p discovers that it was erroneously suspected by process q, p broadcasts an appropriate refutation, refuting the suspicion of process q.

In task 3, when process p receives a refutation message (r, *refutes*, q, k) from process r, it updates $refuted_p(r, q)$ to k.

In task 4, the following is repeatedly done for every process r: if there exists a process q such that the number of times q suspects process r is greater than the number of times the process r refutes q according to p, then process r is added to the suspect list of process p. Otherwise, r is removed from the suspect list of process p.

Correctness argument

A correctness argument of the algorithm is as follows. When a process q receives a suspect message accusing process p, process q may add p to its list of suspects S_q. However, upon receiving p's refutation, process q will remove p from its list of suspects S_q. However, p can be suspected again and added to S_q a second time. However, a further refutation from p will cause p to be again removed from S_q. Thus, a possibly infinite sequence of suspicions followed by corresponding refutations may occur, resulting in p being repeatedly added to and removed from S_q. However, from the eventual weak accuracy property of D, there is a time after which some correct process is not suspected. That is, there is a process p such that there is a time after which no correct process receives a message of type (*, *suspects*, p, k), suspecting p. Thus, after a time no correct process adds process p to its suspect list. Together with the refutation mechanism, this ensures the eventual weak accuracy property of the constructed E.

Now let us see why E satisfies the strong completeness property. Since D satisfies the weak completeness property, eventually every process that crashes is permanently suspected by some correct process, say p. Thus, eventually process p will repeatedly broadcast (p, *suspects*, *, k) messages for these crashed processes and since these processed have crashed, no one will send refute messages for them. Thus all crashed processes will eventually belong to the suspect list of all correct processes. Thus, due to the broadcast of suspect messages and weak completeness property of D, E satisfies the strong completeness property. Thus E satisfies strong completeness and weak accuracy.

15.3 The consensus problem

In the consensus problem, each correct process proposes a value and all processes must reach a unanimous and irrevocable decision on a value that is related to the proposed values [9]. The consensus problem is defined in terms of the following properties:

- **Termination** Every correct process eventually decides some value.
- **Uniform integrity** Every process decides at most once.
- **Agreement** No two correct processes decide differently.
- **Uniform validity** If a process decides a value v, then some process proposed v.

It is widely known that the consensus cannot be solved in asynchronous systems in the presence of even a single crash failure. This is primarily because one cannot distinguish between a process that has crashed and a process that is responding very slow (may be due to the slow network).

15.3.1 Solutions to the consensus problem

Chandra and Toueg [3] showed how to solve the consensus problem using unreliable failure detectors for each of the eight classes of failure detectors. From the following property, the classes of failure detectors P, S, $\diamond P$, $\diamond S$ are, respectively, equivalent to failure detectors ϑ, W, $\diamond\vartheta$, $\diamond W$. Notationally,

$$P \equiv \vartheta, S \equiv W, \diamond P \equiv \diamond\vartheta, \text{ and } \diamond S \equiv \diamond W.$$

So the problem of solving the consensus problem using unreliable failure detectors reduces to solving it for four classes of failure detectors that satisfy strong completeness (i.e., P, S, $\diamond P$ and $\diamond S$), instead of solving it for all eight classes. Since P is reducible to S and $\diamond P$ is reducible to $\diamond S$ (i.e., $P \sqsubseteq S$ and $\diamond$ P $\sqsubseteq\diamond$ S), the algorithms for solving consensus using S also solve the consensus using P and the algorithms for solving consensus using $\diamond S$ also solve the consensus using $\diamond P$.

Next, we present algorithms that solve consensus using S and $\diamond S$. The consensus algorithm using S can tolerate any number of process failures. However, the consensus algorithm using $\diamond S$ requires a majority of the processes to be operational.

15.3.2 A solution using strong failure detector *S*

Algorithm 15.3 solves the consensus problem in an asynchronous system using a failure detector D that satisfies strong completeness and weak accuracy (i.e., $D \in S$). This algorithm tolerates any number of process failures (up to $n-1$ faulty processes among a total of n processes).

The following notation will be used:

- i_p is the value proposed by process p.
- $\perp$ is the null value.
- $V_p[q]$ is the process p's estimate of process q's proposed value.
- V_p is process p's estimate of the proposed values by all other processes.
- $\triangle_p$ contains all the values of V_p.
- r_p is the current round number of process p.
- $msgs_p(r_p)$ is the set of messages that p recieves from other processes about the proposed values in round r_p.
- $lastmsgs_p$ contains the recieved V_q for all processes q by process p.

Every process p executes the following:
procedure propose(i_p)
$V_p \leftarrow <\perp, \perp, \ldots, \perp>$ {p's estimate of the proposed values}
$V_p[p] \leftarrow i_p$;
$\triangle_p \leftarrow V_p$

Phase 1: {Execute round r_p, $1 \leq r_p \leq n-1$}
for $r_p \leftarrow 1$ to $n-1$
  p sends $(r_p, \triangle_p, p)$ to all other processes
  wait until $[\forall q : received\ \ (r_p, \triangle_q, q)$ or $q \in D_p]$ {query the failure detector}
  $msgs_p[r_p] \leftarrow \{(r_p, \triangle_q, q) \mid received(r_p, \triangle_q, q)\}$
  $\triangle_p \leftarrow <\perp, \perp, \ldots, \perp>$
  for $k \leftarrow 1$ to n
    if $(V_p[k] = \perp$ and $\exists (r_p, \triangle_q, q) \in msgs_p(r_p)$ with $\triangle_q[k] \neq \perp)$
    then $V_p[k] \leftarrow \triangle_q[k]$
      $\triangle_p[k] \leftarrow \triangle_q[k]$

Phase 2: p sends V_p to all processes
wait until $[\forall q : received\ V_q$ or $q \in D_p]$ {query the failure detector}
$lastmsgs_p \leftarrow \{V_q \mid\ received\ V_q\}$
for $k \leftarrow 1$ to n
  if $\exists\ V_q \in lastmsgs_p$ with $V_q[k] = \perp$
    then $V_p[k] \leftarrow \perp$

Phase 3: decide on the first non-$\perp$ element of V_p

Algorithm 15.3 An algorithm to solve the consensus problem using a strong failure detector $D \in S$ [3].

An explanation of the algorithm

This algorithm has three phases. Initially, V_p is set to null and $V_p[p]$ contains the value, i_p, proposed by process p.

In the first phase, each process executes $n-1$ asynchronous rounds. In each round, processes broadcast and relay their proposed values. Then, each process p waits until it receives a round r message from every process that is not in D_p, before proceeding to round $r+1$. While p is waiting for a message from a process q in round r, it is possible that q is added to D_p. If this is the case, p does not wait for q's message before it proceeds to round $r+1$. All messages recieved by p in round r_p are stored in $msgs_p(r_p)$. If p's estimate of some process k's proposed value is null and it has recieved a message of the form $(r_p, \triangle_q, q)$ such that q's estimate of process k's proposed value is not null, then p updates its estimate of k's proposed value to q's estimate of process k's proposed value.

In the second phase, a process p broadcasts its estimate of the proposed values of the processes and waits until it receives the estimate from every process that is not in D_p. While p is waiting for an estimate from q, it is possible that q is added to D_p. If this occurs, p stops waiting for q's estimate. By the end of the second phase, correct processes agree on a vector based on the proposed values of all processes. The ith element of this vector either contains the proposed value of process p_i or $\perp$. If any of the correct processes does not agree with the proposed value of a process, say p_i, then the ith element in the vector is set to null and consensus is not reached on the proposed value. It has been shown that this vector contains the proposed value of at least one process.

In the third phase, all correct processes decide the first non-trivial component of this vector.

This solution for the consensus problem using strong failure detectors, even one having weak accuracy property, has an excellent fault tolerance capacity; the solution tolerates any number of process failures.

Also, since a weak failure detector W is reducible to a strong failure detector S using the algorithm given above, this algorithm also solves the consensus using a weak failure detector W.

15.3.3 A solution using eventually strong failure detector $\diamond S$

The previous solution to the consensus problem used failure detectors with *weak accuracy*: some correct process is never suspected. We now present a solution to the consensus problem using a failure detector that satisfies the *eventual weak accuracy*: all processes may be erroneously added to the lists of suspects at one time or another, but there is a time after which a correct process p is permanently removed from the list of suspects. However, at any given time t, processes cannot determine if a particular process is correct, or whether a correct process will never be suspected after time t.

Algorithm 15.4 presents a solution to the consensus using an eventually strong failure detector $D \in \diamond S$. Such failure detectors satisfy strong completeness

---

Every process p executes the following:
$estimate_p \leftarrow i_p$ {p's estimate of the decision value}
$state_p \leftarrow$ undecided
$r_p \leftarrow 0$ {r_p denotes the current round number}
$ts_p \leftarrow 0$ {the round in which $estimate_p$ was last updated, initially 0}
cobegin
||Task 1: {Rotate through coordinators until a decision is reached}
while $state_p =$ undecided
  $r_p \leftarrow r_p + 1$
  $c_p \leftarrow (r_p \ mod \ n) + 1$ {c_p is the current coordinator}

Phase 1: {All processes p send $estimate_p$ to the current coordinator}
  p sends $(p, r_p, estimate_p, ts_p)$ to c_p

Phase 2: {The current coordinator gathers $\lceil (n+1)/2 \rceil$ estimates and proposes a new estimate}
  if $p = c_p$ then
    wait until [for $\lceil (n+1)/2 \rceil$ processes q: $received(q, r_p, estimate_q, ts_q)$ from q]
    $msgs_p[r_p] \leftarrow \{(q, r_p, estimate_q, ts_q) \mid p\ received(q, r_p, estimate_q, ts_q)$ from $q\}$
    $t \leftarrow$ largest ts_q such that $(q, r_p, estimate_q, ts_q) \in msgs_p[r_p]$
    $estimate_p \leftarrow$ select one $estimate_q$ such that $(q, r_p, estimate_q, t) \in msgs_p[r_p]$
    p sends $(p, r_p, estimate_p)$ to all processes

Phase 3: {All processes wait for the new estimate proposed by the current coordinator}
  wait until $[received(c_p, r_p, estimate_{c_p})$ from c_p or $c_p \in D_p]$ {Query the failure detector}
  if $[received(c_p, r_p, estimate_{c_p})$ from $c_p]$ then {p received $estimate_{c_p}$ from c_p}
    $estimate_p \leftarrow estimate_{c_p}$
    $ts_p \leftarrow r_p$
    p sends (p, r_p, ack) to c_p
  else
    p sends $(p, r_p, nack)$ to c_p {p suspects that c_p crashed}

Phase 4: {The current coordinator waits for $\lceil (n+1)/2 \rceil$ replies. If these replies indicate that $\lceil (n+1)/2 \rceil$ processes adopted its estimate, the coordinator broadcatss a request to decide.}
if $p = c_p$ then
  wait until [for $\lceil (n+1)/2 \rceil$ processes q: $received(q, r_p, ack)$ or $(q, r_p, nack)$]
  if [for $\lceil (n+1)/2 \rceil$ processes q: $received(q, r_p, ack)$]
    then p R-broadcasts $(p, r_p, estimate_p, decide)$

||Task 2: {When p receives a $decide$ message, it decides}
  when p R-delivers $(q, r_q, estimate_q, decide)$ for some q
  if $state_p = undecided$ then
    $decide$ on $estimate_q$
    $state_p \leftarrow$ decided
  coend

Algorithm 15.4 An algorithm to solve the consensus problem using an eventually strong failure detector $D \in \diamond S$ [3].

and eventual weak accuracy. The algorithm requires that a majority of the processes are always up. If f is the maximum number of processes that may crash at any time, this algorithm requires that $f < \lceil n/2 \rceil$, that is, at least $(n+1)/2$ processes are correct at all times.

An explanation of the algorithm

This algorithm proceeds in asynchronous rounds and makes use of the *rotating coordinator* paradigm until a decision is reached. All processes know that during round r, the coordinator is process $c = (r \bmod n) + 1$. All messages are either to or from the "current" coordinator. The "current" coordinator tries to determine a consistent decision value. If the current coordinator is correct and is not suspected by any surviving process, then it succeeds and broadcasts the decision value.

The algorithm goes through three asynchronous stages where each stage can contain several asynchronous rounds. In the first stage, several decision values are proposed. In second stage, a value gets locked: no other decision value is possible. In the third and final stage, the processes decide on the locked value and consensus is reached.

Initially, the state of a process p is "undecided" and its estimate of the decision value is i_p. A timestamp ts_p is associated with every process p that contains the round number when its estimate was last updated.

Each round of task 1 consists of four asynchronous phases. In phase 1, every process p sends its current estimate of the decision value to the current coordinator c_p. It also sends the round number (ts_p) in which it adopted this estimate.

In phase 2, the coordinator c_p gathers $\lceil (n+1)/2 \rceil$ such estimates and proposes a new estimate. The current coordinator waits until it receives estimates from $\lceil (n+1)/2 \rceil$ processes. It stores all these estimates in the array $msgs_p[r_p]$, selects one with the largest timestamp, and sends it to all the processes as the new estimate, $estimate_p$.

In phase 3, all processes wait for the new estimate proposed by the current coordinator. For each process p, there are two possibilities:

- Process p receives $estimate_{c_p}$ from the coordinator c_p: in this case, p updates its timestamp to the current round number and sends an *ack* to c_p to indicate that it adopted $estimate_{c_p}$ as its own estimate.
- Process p does not receive an $estimate_{c_p}$ from the coordinator c_p and, upon consulting its failure detector module D_p, suspects that the coordinator c_p has crashed: in this case, p sends a *nack* to c_p.

In phase 4, the coordinator c_p waits for $\lceil (n+1)/2 \rceil$ replies (*acks* or *nacks*). If all replies are *acks*, then c_p knows that a majority of processes changed

their estimates to $estimate_{c_p}$ and thus $estimate_p$ is locked and c_p broadcasts a request to decide value $estimate_p$.

In task 2, at any time, if a process receives such a request, it decides accordingly, i.e., when a process p receives a message of the form (q, r_q, $estimate_q$, *decide*) from a process q, then p decides on the estimate of q provided it has not already decided. In this case, process p changes its state to "decided."

For correctness of the algorithm, we have to show that the algorithm satisfies termination, uniform validity, agreement, and uniform integrity properties. The readers are referred to the original source for a correctness proof.

This algorithm requires that $f < \lceil n/2 \rceil$, i.e., at least $\lceil n/2 \rceil$ processes are correct, and assumes that processes have a priori knowledge of the list of potential coordinators.

15.4 Atomic broadcast

Atomic broadcast is one of the fundamental problems in fault-tolerant distributed computing. It is a powerful paradigm in the design of fault-tolerant distributed computing systems. Chandra and Toueg [3] showed that the results of consensus can be applied to solve the problem of atomic broadcast. Informally, atomic broadcast requires that all correct processes deliver the same set of messages in the same order (i.e., deliver the same sequence of messages). Formally, *atomic broadcast* can be defined as a reliable broadcast with the total order property.

The total order property If two correct processes p and q deliver two messages m and m', then p delivers m before m' if and only if q delivers m before m'.

The total order and agreement properties of atomic broadcast ensure that all correct processes deliver the same sequence of messages.

In asyncronous sytems with crash failures, consensus and atomic broadcast are equivalent and this can be shown by reducing one to the another. Consensus can be reduced to atomic broadcast as follows: In the consensus problem, to propose a value, a process atomically broadcasts it. To decide a value, a process picks the value of the first message that it atomically delivers. The total order property of atomic broadcast ensures that all correct processes deliver the same first message. Hence, all correct processes choose the same value and the agreement property of the consensus is satisfied. In the next section, we show how to reduce atomic broadcast to consensus.

A consequence of this equivalence is that a solution for one can be used to solve the other. In addition, it implies the following for solving atomic broadcast in asynchronous systems:

- Since consensus has no deterministic solution in aynchronous systems, even if we assume that at most one process may fail by crashing, atomic broadcast cannot be solved by a deterministic algorithm even if at most one process may fail by crashing.
- As consensus is solvable using randomization or unreliable failure detectors in asynchronous systems, atomic broadcast can be solved using these techniques.

15.5 A solution to atomic broadcast

Algorithm 15.5 presents a solution (due to Chandra and Toueg [3]) to atomic broadcast problem using the consensus in asynchronous systems. This algorithm shows how to transform any consensus algorithm into an atomic broadcast algorithm in asynchronous systems. This atomic broadcast algorithm tolerates as many faulty processes as the consensus algorithm does.

This atomic broadcast algorithm uses repeated executions of consensus. The kth execution of consensus is used to decide on the kth batch of messages to be atomically delivered. Processes distinguish between these executions by tagging all the messages pertaining to the kth execution of consensus with the counter k.

The atomic broadcast algorithm uses $R_broadcast(m)$ and $R_deliver(m)$ primitives of reliable broadcast. To avoid any confusion, note that the primitives $A_broadcast(m)$ and $A_deliver(m)$ respectively refer to a broadcast and a delivery in atomic broadcast, while primitives $R_broadcast(m)$ and $R_deliver(m)$ respectively refer to a broadcast and a delivery associated with reliable broadcast. $propose(k, -)$ and $decide(k, -)$ are the propose and decide primitives corresponding to the kth execution of consensus.

An explanation of the algorithm

The algorithm consists of three tasks such that: (i) a task that is enabled is eventually executed and (ii) a task i can execute concurrently with another task j provided $i \neq j$.

In task 1, when a process p wants to A-broadcast a message m, it R_broadcasts m. In task 2, a message m is added to set $R_delivered_p$ when process p R_delivers it.

In task 3, when a process p A_delivers a message m, it adds m to set $A_delivered_p$. $A_undelivered_p$ (defined as $R_delivered_p - A_delivered_p$) is the set of messages that p has R_delivered but has not A_delivered yet. Process p periodically checks whether $A_undelivered_p$ contains messages. If $A_undelivered_p$ contains messages, p enters its next execution of consensus, say the kth one, and proposes $A_undelivered_p$ as the next batch of messages to be A_delivered. Process p then waits for the kth consensus decision, which is denoted by $msgSet^k$. $msgSet^k$ contains messages that are R_delivered but are

Every process p executes the following:

Initialization:

$R_delivered \leftarrow \emptyset$
$A_delivered \leftarrow \emptyset$
$k \leftarrow 0$

To execute $A_broadcast(m)$: {Task 1}
$R_broadcast(m)$

$A_deliver(-)$ occurs as follows:
when $R_deliver(m)$ {Task 2}
  $R_delivered \leftarrow R_delivered \bigcup \{m\}$
when $R_delivered - A_delivered \neq \emptyset$ {Task 3}
  $k \leftarrow k+1$
  $A_undelivered \leftarrow R_delivered - A_delivered$
  $propose(k, A_undelivered)$
  wait until $decide(k, msgSet^k)$
  $A_deliver^k \leftarrow msgSet^k - A_delivered$
  atomically deliver all messages in $A_deliver^k$ in some determinisic order
  $A_delivered \leftarrow A_delivered \bigcup A_deliver^k$

Algorithm 15.5 A Solution to atomic broadcast using consensus [3].

yet to be A_delivered. Finally, p A_delivers all the messages in $msgSet^k$ except those already A_delivered by it (i.e, all the messages in the set $A_deliver_p^k = msgSet^k - A_delivered_p$) in some deterministic order that was agreed *a priori* by all processes.

For a correctness proof of the algorithm, the readers should refer to the original source.

15.6 The weakest failure detectors to solve fundamental agreement problems

Delporte-Gallet *et al.* [5] showed that, if we exclude unrealistic failure detectors,[1] then in an environment where we do not bound the number of faulty processes, the class of perfect failure detectors P is the weakest to solve fundamental agreement problems like uniform consensus, atomic broadcast, and terminating reliable broadcast (also called the Byzantine Generals).

[1] Unrealistic failure detectors are failure detectors that can guess the future and thus cannot be implemented even in a perfectly synchronous systems.

Delporte-Gallet *et al.* [5] collapsed the Chandra–Toueg failure detector hierarchy in this environment, and showed that P is the only useful class to solve these agreement problems. This explains why most known reliable distributed systems rely on a group membership service that precisely aims at emulating a perfect failure detector P, that is, when a process is suspected due to a timeout, it is excluded from the group. Thus, every suspicion is taken as being accurate.

Uniform consensus

In consensus, the agreement property allows the bad processes to decide differently from good processes. This fact can be sometimes undesirable as it does not prevent a bad process from propagating a different decision in the system before crashing. In the uniform consensus, the uniform-agreement property allows no two processes (good or bad) to decide differently, which enforces the same decision on any process that decides.

Terminating reliable broadcast

Solving the consensus problem is equivalent to solving the atomic broadcast problem, in any system with reliable channels (i.e., where only a finite number of messages can be lost). Atomic broadcast entails delivering messages to processes in a reliable and totally ordered manner. Terminating reliable broadcast is a stronger form of atomic broadcast. In *terminating reliable broadcast*, the processes deliver messages in the same sequence as atomic broadcast does, but, in addition, processes should deliver a specific nil value for every message that was broadcast by a faulty process but was not delivered by any correct process. This problem is a rephrasing of the famous Byzantine Generals problem in the fail-stop model.

Delporte-Gallet *et al.* [5] showed that, in environments where the number of faulty processes is not bounded, uniform consensus is strictly harder than consensus, and uniform consensus and atomic broadcast are strictly weaker than terminating reliable broadcast.

In environments where the number of faulty processes is not bounded, the exact information about failures needed to solve consensus (hence atomic broadcast) and terminating reliable broadcast, is captured by P. Thus, in the failure detector hierarchy, P is the only useful class to solve the agreement problems.

15.6.1 Realistic failure detectors

Note that a failure detector has been defined as *any* function of the failure pattern and this function may be able to provide information about the future failures. Such a failure detector does not factor out synchrony assumptions of the system and cannot be implemented even in a perfectly synchronous system.